

어떤 문제를 풀려는가

어려운 Study paper 를 그림과 예로 다시 읽는 안내서

읽는 법 이 문서는 원 Study 의 결론을 바꾸지 않고, 질문→세 문장→직관→예/반례→그림→대안→미증명 지점 순서로 풀어 쓴다. 파란색 training 용어는 별도 Background paper 의 해당 개념으로 연결된다. 주장 ID 와 figure ID 는 원 Study 와 동일하다.

Contents

E02 · 어떤 문제를 풀려는가	2
이 권이 답할질문	2
먼저 세 문장	2
직관	2
예와 반례	3
그림 읽기	3
대안과 비교	4
아직 모르는것	5
더 깊이 읽기	5
문서 경계와 사용법	6

E02 · 어떤 문제를 풀려는가

이 권이 답할 질문

Sleep-time compute 가 등장한 이유를 “AI 도 사람처럼 자야 해서” 라고 설명하면 문제를 놓친다. 실제 출발점은 배포된 agent 가 장기간 경험을 쌓을 때 생기는 다섯 병목이다: **context overflow, state staleness, catastrophic forgetting, finite capacity, governance debt**. 이 중 일부는 retrieval 이나 cache 로 해결되고, 일부는 background consolidation 이 유리하며, 일부는 아직 어떤 방식으로든 해결되지 않았다.

이 권은 문제분해를 바탕으로 **STC-C032** · author_claim · confidence high, **STC-C033** · synthesis · confidence high, **STC-C035** · author_claim · confidence high, **STC-C037** · synthesis · confidence high 을 설명한다.

먼저 세 문장

1. Long context 는 “많이 넣는 능력” 이지 “무엇을 오래 보존하고 갱신할지 결정하는 능력” 이 아니다. context window 가 커져도 오래된 episode 의 충돌 · 삭제 · 중요도 문제는 남는다.
2. 지속적으로 weight 나 memory 를 갱신하면 유한한 용량 예산 안에서 새 지식이 옛 지식의 접근 가능성을 손상시킬 수 있다. 정보가 parameter 안에 남아 있어도 행동으로 꺼내지 못하면 실용적으로는 잊은 것이다.
3. Sleep 은 여러 episode 를 batch 로 비교하고 정리할 기회를 주지만, synthetic replay 의 오류 · memory poisoning · 삭제 전파라는 새 문제도 만든다.

직관

문제 1 — Context 는 작업대이지 창고가 아니다

Context window 는 현재 reasoning 에 필요한 자료를 펼쳐 놓는 작업대다. 작업대가 커지면 더 많은 문서를 동시에 볼 수 있지만, 매번 모든 과거를 다시 펼치는 비용은 sequence 길이에 따라 증가한다. 또한 중요하지 않은 기록이 signal 을 가리고, 서로 충돌하는 사실의 시간 순서를 모델이 잘못 읽을 수 있다. 그래서 long context 와 long-term memory 는 보완 관계이지 대체 관계가 아니다.

문제 2 — 기억은 저장보다 선택이 어렵다

새 episode 를 모두 저장하면 memory 는 단조 증가한다. 검색 후보가 늘면서 index 비용, false retrieval, compaction 비용, privacy exposure 가 함께 증가한다. 반대로 aggressive summary 는 중요한 tail detail 을 잃는다. 어떤 record 를 admit · merge · supersede · evict 할지 결정하는 정책이 필요하다.

문제 3 — Weight 에는 주소가 없다

외부 DB 에서는 “이 문장은 어느 대화에서 왔는가” 를 pointer 로 남길 수 있다. Weight update 는 많은 example 을 gradient 로 섞기 때문에 특정 사실이 어느 parameter 에 얼마나 들어갔는지 역추적하기 어렵다. 삭제 요청이 오면 adapter 전체를 버리거나 원 dataset 에서 재학습해야 할 수도 있다. 따라서 parametric memory 는 빠른 access 를 얻는 대신 provenance 와 surgical delete 를 잃기 쉽다.

문제 4 — Continual learning 은 retention 과 plasticity 의 줄다리기다

지속학습은 새 경험에 적응해야 하지만 옛 능력을 지켜야 한다. update 를 크게 하면 plastic 하지만 망각이 커지고, 작게 하면 안정적이지만 새 정보가 안들어간다. 이를 파국적 망각과 stability-plasticity dilemma 라고 부른다. 단순히 parameter 를 늘리면 시간을 벌 수 있지만 capacity 와 serving cost 가 계속 자란다.

직관

도서관의 진짜 병목은 책장을 더 사는 일이 아니라, 중복판을 합치고 낡은 정보를 표시하고 출처를 남기며 필요한 책을 제때 찾는 일이다.

예와 반례

예 — “내 주소” 가 세 번 바뀐 사용자

2025 년, 2026 년 봄, 2026 년 여름 주소가 서로 다른 세 episode 에 있다. Naive vector search 는 query 와 가장 비슷한 과거 주소를 가져올 수 있다. Temporal graph 는 valid-time 과 transaction-time 을 붙여 최신 주소를 고를 수 있다. Summary 는 “현재 주소” 만 남겨 간단하지만, 왜 바뀌었는지와 과거 배송분쟁을 읽을 수 있다. Weight update 는 답을 빠르게 만들 수 있지만 어떤 주소가 학습되었는지 검증·삭제가 가장 어렵다.

예 — 새로운 error code 의 반복학습

공장 장비의 새 error code 가 매일 수백 번 등장한다. 단기에는 external memory 에 원 log 와 복구 절차를 저장하는 편이 안전하다. 사용 빈도와 안정성이 확인되면 domain adapter 로 승격할 수 있다. 이후 제조사 문서가 수정되면 adapter 를 rollback 하고 다시 build 해야 한다. 이 흐름은 sleep 을 “기억 저장” 이 아니라 **promotion pipeline** 으로 본다.

반례 — Long context 만능리기

과거 1 년의 모든 log 를 million-token context 에 넣을 수 있다고 해도 매 요청마다 읽을 이유는 없다. 비용·latency·irrelevant evidence 가 늘고, 삭제요청이 반영됐는지 확인하기도 어렵다. Long context 가 유용한 것은 선택된 evidence 를 깊게 읽을 때다. 선택·정리는 여전히 memory lifecycle 의 일이다.

반례 — 모든 것을 nightly replay

Generated replay 가 과거 distribution 을 완벽히 대표한다고 가정하면 현실을 과도하게 단순화한다. 생성 모델은 rare tail 을 빼먹거나 자기 오류를 반복할 수 있다. **STC-C035** · author_claim · confidence high 는 real-data support 없이 synthetic data 를 재귀적으로 학습할 때 distribution tail 이 붕괴할 수 있음을 경고한다.

그림 읽기

행은 해결해야 할 문제, 열은 대안이다. Long context 는 immediate access 에는 강하지만 durable update 와 delete 에는 약하다. External memory 는 provenance-rollback 이 강하지만 retrieval 과 context cost 가 있다. Adapter/weight 는 access latency 를 줄일 수 있지만 interference 와

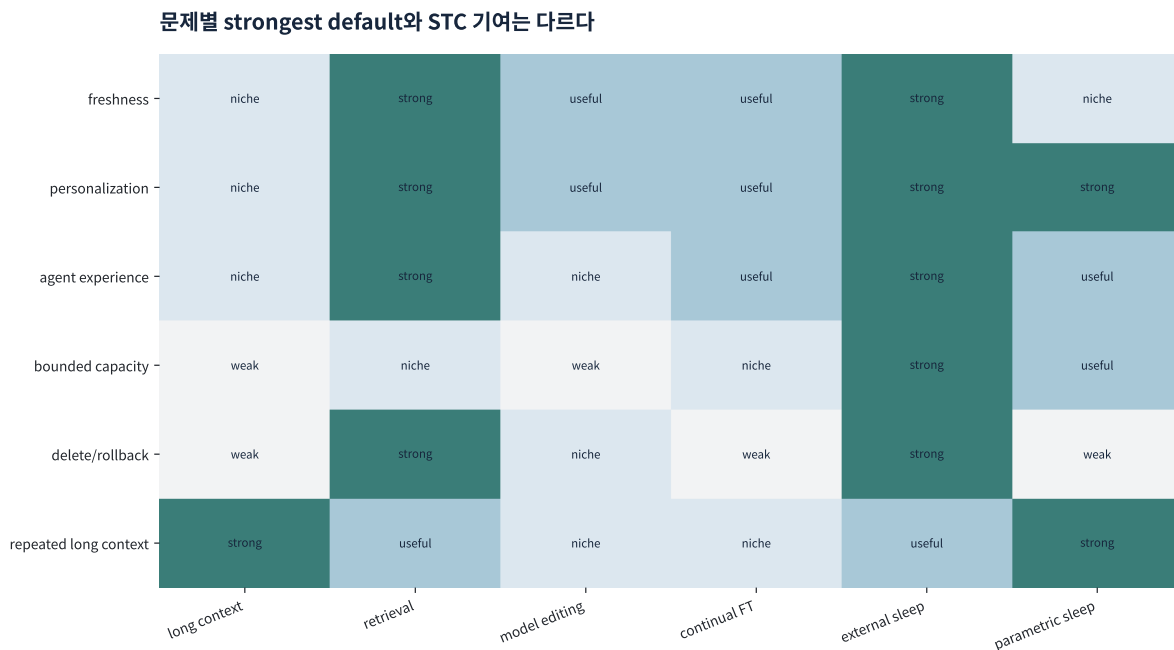


Figure 1: STC-F004. Problem-to-solution comparison matrix. 원 Study 의저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

governance 가 어렵다. Sleep 은 독립 열이 아니라 이 방법들을 응답후 batch 로 조합하는 lifecycle 이다.

가로축은 memory load, 세로축은 retained utility 로 읽는다. 초반에는 memory 를 늘릴수록 utility 가 빠르게 증가하지만 어느 지점부터중복·간섭·검색 noise 가 커져 marginal utility 가 꺾인다. 이 “capacity knee” 는 이미 보편 law 로 입증된 값이 아니라 연구 가설이다. 중요한 것은 device capacity 만 늘리는 것이 아니라 knee 의 위치를 admission·compaction·tiering 으로 옮기는 것이다.

대안과 비교

근본 문제	sleep 이 제공하는 것	sleep 없이 가능한 강한 대안	판단 기준
context overflow	응답 후 요약·index rebuild	retrieval, recurrent state, compression	answer quality / token·latency
stale/conflicting facts	batch conflict resolution	temporal KG, system-of-record	temporal accuracy / delete
forgetting	replay·distillation·isolation	EWC, GEM, adapters, expansion	retained utility / interference
finite memory	compaction·promotion·eviction	rate-distortion policy	utility per byte / tail loss
governance	versioned sleep output	immutable event log	provenance / rollback time

Sleep 이 특히 유리한 곳은 한 episode 만 봐서는 판단할 수 없는일이다. 여러 기록에서 stable preference 를 찾아내거나, conflict 를 시간순으로 해결하거나, reuse 가 충분한 지식을 adapter 로 승격하는 일이다. 즉시반응이 중요하거나 source-of-truth 가 명확한 factual lookup 에는 external retrieval 이 더 단순하고 안전하다.

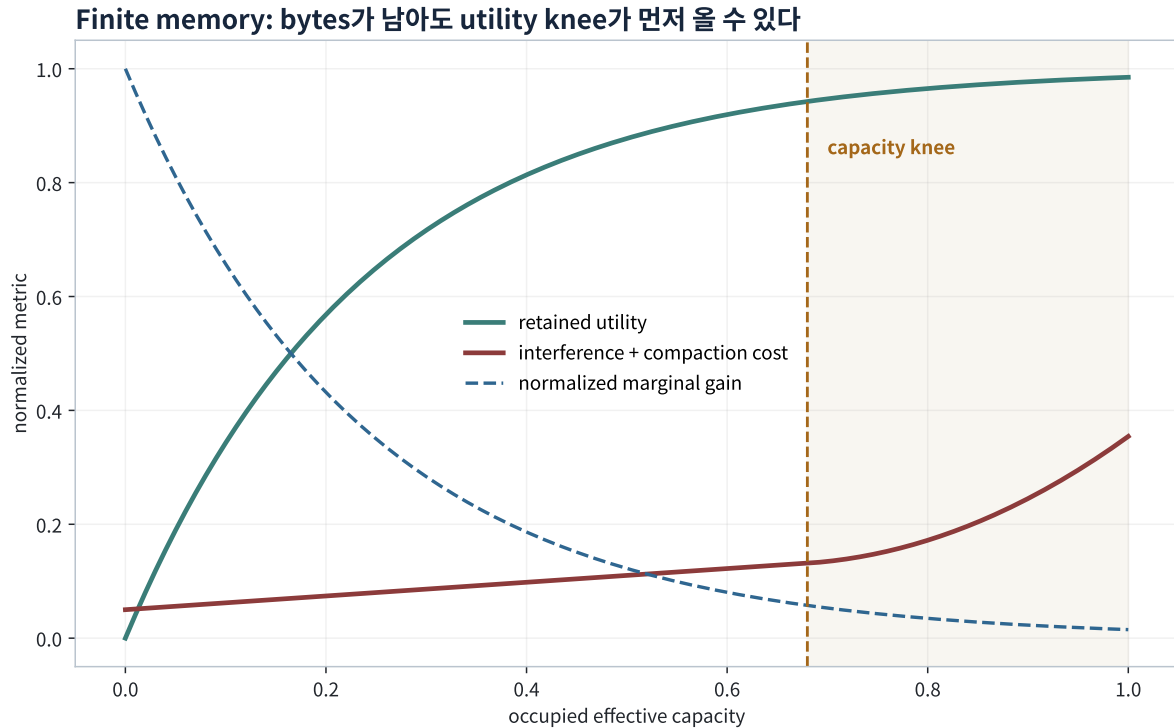


Figure 2: STC-F009. Finite-memory capacity knee. 원 Study 의 저자 작성합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

아직 모르는것

- Static memorization capacity 는 lifelong safe capacity 와 다르다. sequential interference·retrieval failure·delete overhead 를 함께 재는방법이 필요하다.
- Weight 안에 정보가 “encoded” 되었다는 probe 결과와 실제 answer 에서 “accessible” 하다는 결과 사이의 gap 을 어떻게 측정할지 합의가 없다.
- Synthetic replay 에서 real-data anchor 를 얼마나 유지해야 tail collapse 를 막는지 task 별 scaling law 가 없다.
- External memory poisoning 과 parametric poisoning 을 같은 benchmark 에서비교한 공개 증거가 부족하다.
- Memory capacity knee 가 workload, user 수, record entropy, device tier 에 따라 어떻게 이동하는지 측정되지 않았다.

주의

문제의 원인은 memory byte 부족 하나가 아니다. 잘못된 기억을 오래 · 빠르게제공하는 시스템은 큰 memory 를 가진 실패한 시스템이다.

더 깊이 읽기

- 원 분석: 다섯문제의 분해와 solution boundary
- Training Background: 지속학습, 파국적망각, 용량예산, 출처추적성, 일반화
- Claim route: **STC-C032** · author_claim · confidence high · **STC-C033** · synthesis · confidence high · **STC-C035** · author_claim · confidence high · **STC-C037** · synthesis · confidence high

문서 경계와 사용법

이 문서는 쉬운 설명본이지 새로운 증거원이 아니다. 수치 · 인용 · claim status 는 원 Study paper 와 claim map 이 우선한다. 학습 개념은 Training Background paper 를 함께 읽고, 논문 재현이나 외부 배포 판단에는 고정 source 와 rights audit 를 사용한다.